Exkurs W

Werkzeuge

Agenda

Abschnitt W.1

Versionsverwaltung mit Git

Abschnitt W.2

Build Management mit Apache Maven

Abschnitt W.3

Komponententests mit JUnit

Klausurrelevante Lernziele

- Komponenten einer Git-Arbeitsumgebung erläutern können
- Lebenszyklus von Dateien in Git kennen
- Git-Befehle »clone«, »status«, »add«, »commit«, »fetch«, »pull« und »push« kennen und anwenden können
- Einen einfachen Komponententests mit JUnit erstellen und das Namensschema für Testklassen und Testmethoden anwenden können
- Nicht klausurrelevant
  - Andere Git-Befehle

Abschnitt W.1

Versionsverwaltung mit Git

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Git

Linus Torvalds
(* 1969)

- Freie Software für die verteilte Versionsverwaltung
- 2005 von Linus Torvalds initiiert
- »Blödmann« in britischer Umgangssprache
- Internationaler De-Facto-Standard
  - Stack Overflow: ca. 94% aller Entwickelnden nutzen Git (Developer Survey 2022)
  - Verwaltung Linux-Kernel & Microsoft Windows (seit 2017)
  - Grundlage von GitHub, GitLab, Bitbucket, Azure DevOps
- Kommandozeilen-Werkzeug (Command Line Interface, CLI)
  - GUI-Erweiterungen & Integration in IDE, z.B. Eclipse, Visual Studio, Visual Studio Code etc.
- Git speichert nicht die Änderungen, sondern Snapshots der Dateien

Repository (aka Repo) – Collection of all software-related artifacts belonging to a system.
(ISO/IEC/IEEE 24765:2017)

Quelle: (Stack Overflow 2022)

Zusätzliche Quellen

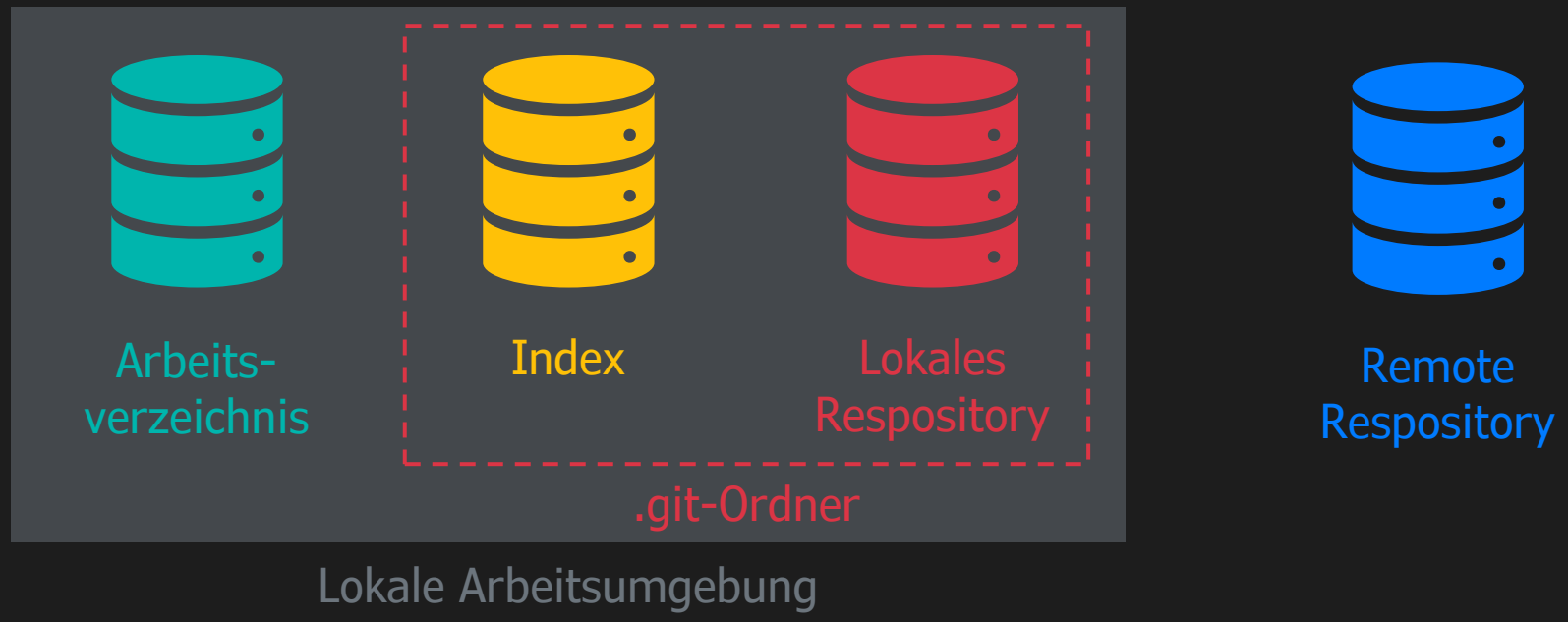
- Download & Dokumentation: https://git-scm.com
- Cheat Sheet: https://git-scm.com/cheat-sheet.html
- Eingebaute Hilfe in der Kommandozeile
  - `git help`
  - `git help <command>`
  - `git help tutorial`
- Tutorial: https://www.w3schools.com/git/
- Interaktive Lernspiele
  - https://www.gitmastery.me
  - https://learngitbranching.js.org

Quelle: Randall Munroe, https://xkcd.com/1597/

Befehle: Erste Schritte

- `git config <name> <value>`
  - Git-Konfigurationen für das aktuelle Repository setzen, insbesondere
    - `git config user.name "Philipp Rohde"`
  - Konfigurationen auflisten
    - `git config --list`
  - Schalter `--global`, um die Konfiguration für alle Repository des Nutzers zu
- `git init <directory>`
  - Erstellt neues Verzeichnis mit angegebenen Namen als neues lokales Repository
  - Alternativ: ohne Angabe des Namens für das aktuelle Verzeichnis
- `git clone <repository>`
  - Klonen eines bestehenden Repository in ein neues Arbeitsverzeichnis
  - Typischerweise ein Remote-Repository, kann aber auch anderes lokales Repository sein
- Datei `.gitignore`: ignorierte Dateien / Dateitypen, siehe https://gitignore.io

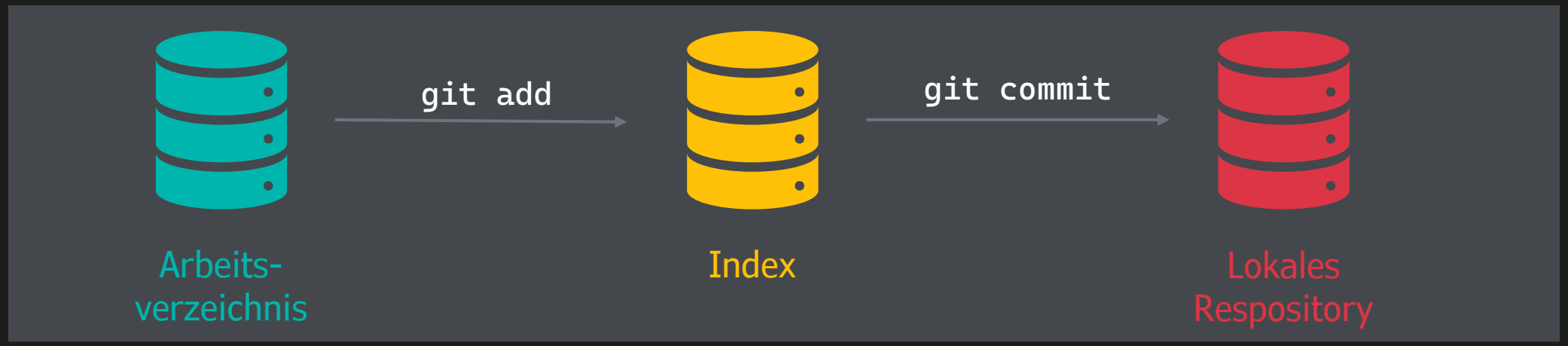
Komponenten



- Arbeitsverzeichnis (aka Working Tree): Dateisystem als lokale Arbeitskopie; hier entstehen die eigentlichen Änderungen
- Index (aka Staging Area): geänderte Dateien, die ins lokale Repository per Commit übernommen werden können
- Lokales Repository: alle Versionen, die per Commit tatsächlich übertragen wurden
- Remote Repository (aka Blessed Repository): gemeinsame Versionen des Teams

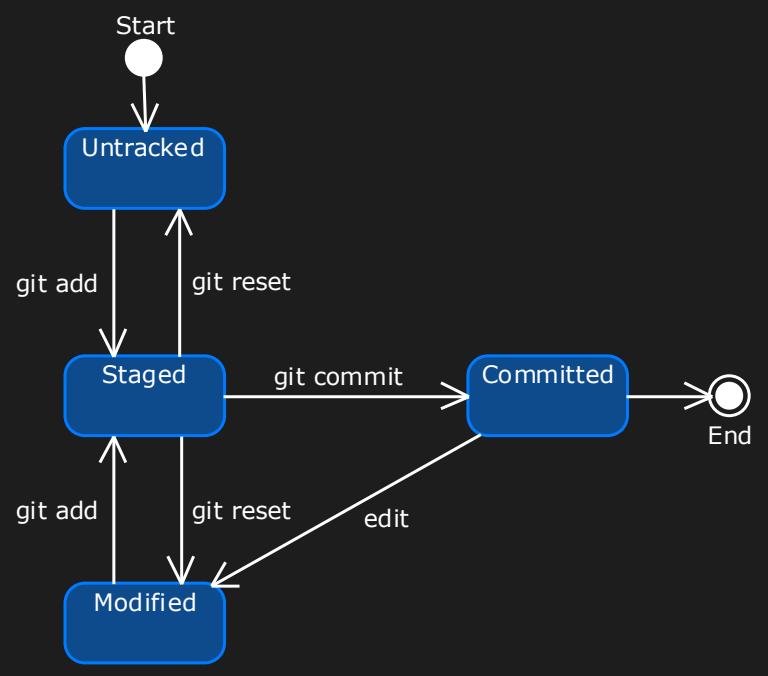
Index versus lokales Repository

- Zwischenschicht: »vorgemerkte« Dateien für nächsten Commit
  - Dateien müssen explizit zum Index hinzugefügt werden, damit sie beim nächsten Commit ins lokale Repository übertragen werden
- Logische Strukturierung von Inhalten: zusammengehörende Dateien werden in gemeinsamen Commit zusammengefasst
- Ermöglicht höhere Sicherheit, um kritische Dateien von Commits auszunehmen



Befehle: Index & Commit

- `git status`
  - Status & Hinweise für lokales Repository anzeigen
- `git add <file>`
  - Datei im Zustand »Untracked« bzw. »Modified« zum Index hinzufügen (Zustand »Staged«)
  - `git add .` um alle Dateien im Zustand »Untracked« bzw. »Modified« hinzuzufügen
- `git rm <file>`
  - Datei aus Arbeitsverzeichnis & Index löschen
- `git mv <old> <new>`
  - Datei im Arbeitsverzeichnis & Index verschieben
- `git commit -m "<message>"`
  - Änderungen aus Index ins lokale Repository mit angegebener Nachricht übertragen
  - `-am` statt `-m`: inkl. Add für alle »Modified«-Dateien im Arbeitsverzeichnis (aber ohne »Untracked«-Dateien)



Lebenszyklus von Dateien in git

Commits

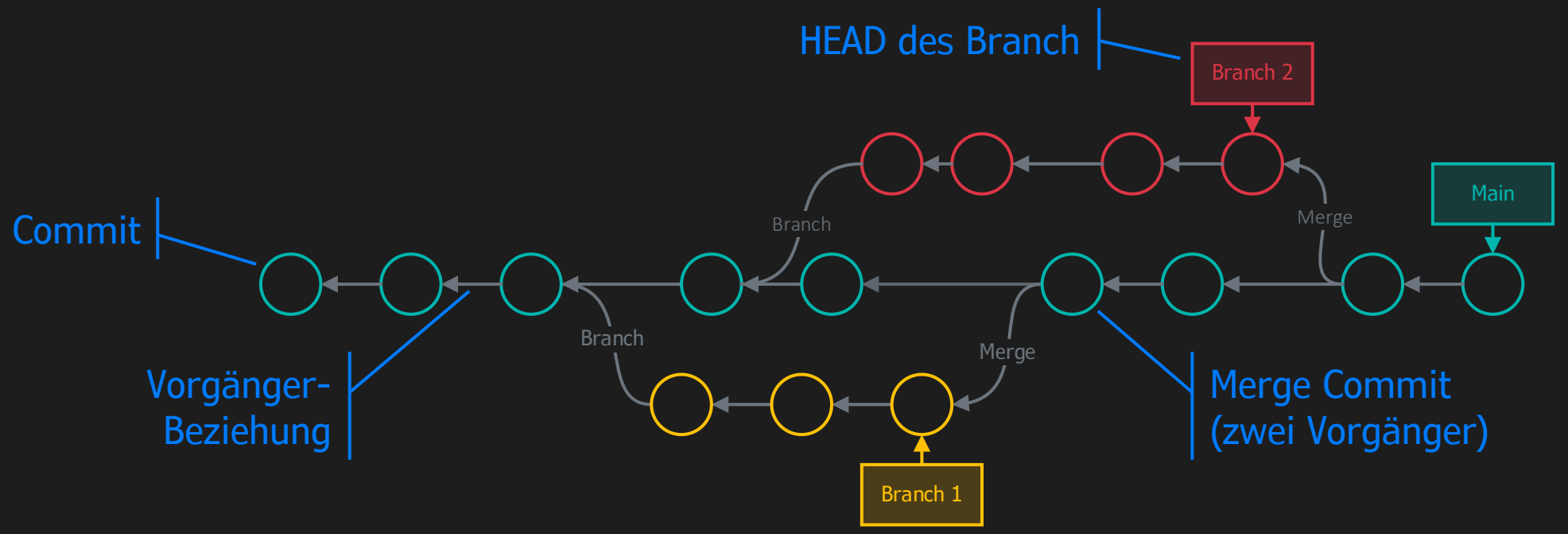
- Commit ist bestätigter »Schnappschuss« des Projekts zu einem bestimmten Zeitpunkt (als Kopie von Dateien)
- Jeder Commit hat eine eindeutige ID (HEX-Hashwert, Länge: 40); die ersten sieben Zeichen sind der »Short Hash«
- Jeder Commit hat eine Referenz auf seinen Vorgänger-Commit (Ausnahmen: erster Commit und »Merge Commits«)
- Dadurch eindeutiger Commit-Verlauf
- Wichtig: aussagekräftige Commit Messages
  - DO'S: kurze, aber aussagekräftige Beschreibung des Bereichs und des Grundes der Änderungen, z.B. »Add implementation of X« oder »Refactor subsystem Y for readability«
  - DONT'S: »Some fixes«, »More bug fixes«; geänderte Dateien aufzählen

Quelle: Randall Munroe, https://xkcd.com/1296/

Referenzen

- In vielen Befehlen können bestimmte Commits oder Branches angegeben werden
- Verschiedene Arten von Referenzen
  - Lokaler Branch: z.B. `main` oder `feature/new-feature`
  - Tag: z.B. `v3.14.1`
  - Commit-ID (Full oder Short): z.B. `be95a90`
  - Remote Branch: z.B. `origin/main`
  - Aktueller Commit des aktuellen Branch: `HEAD`
  - Drei Commits vor aktuellem Commit des aktuellen Branch: `HEAD^^^` oder `HEAD~3`
- Beispiel: `git diff HEAD^` zeigt den Unterschied zwischen dem Arbeitsverzeichnis und dem Vorgänger des aktuellen Commit des aktuellen Branch

Commit-Verlauf als gerichteter Graph



Lokales versus Remote Repository

- Lokale Repository ist die bestätigte Version der Dateien des Entwickelnden und nur für diese Person sichtbar
- Remote Repository ist die bestätigte Version für alle im Team
  - Remote Repository sollte sehr sorgfältig gepflegt werden
  - »Breaking Changes« (z.B. durch Änderungen von zurückliegenden Commits) müssen unbedingt vermieden werden
- Typischer Ablauf
  - Entwickelnde Person führt viele kleine Commits im lokalen Repository aus
  - Nach Bedarf können daraus frühere Zustände wiederhergestellt werden, z.B. wenn ein Versuch nicht funktioniert, die Tests fehlschlagen etc.
  - Wenn entwickelnde Person mit Ergebnis des lokalen Repository zufrieden ist, dann sollte dieses aufgeräumt werden: einzelne Commits auswählen, Reihenfolge der Commits ändern, mehrere Commits zusammenfassen, Rebase etc.
  - Erst dann sollte das lokale Repository per Push in das Remote Repository übertragen werden

Status mit Änderungen

```
A:\SWT [main ≡ +1 ~1 -1 !]> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    DeletedFile.txt
        modified:   ModifiedFile.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        NewFile.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

»origin« ist Referenz auf das Remote-Repository

Änderungen im Arbeitsverzeichnis, aber noch nicht im Index

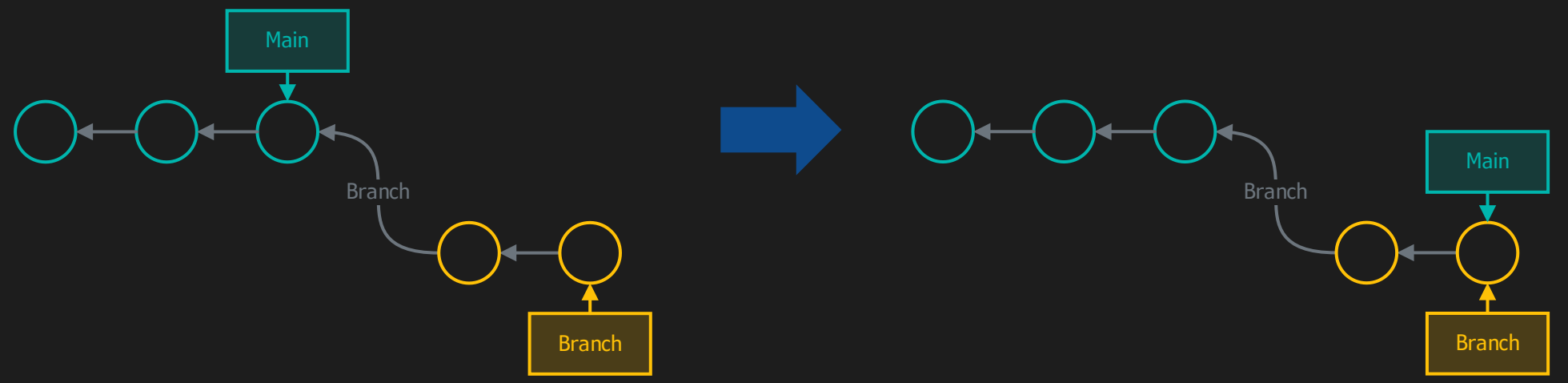
Add & Commit

```
A:\SWT [main ≡ +1 ~1 -1 !]> git add .
A:\SWT [main ≡ +1 ~1 -1 ~]> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    DeletedFile.txt
        modified:   ModifiedFile.txt
        new file:   NewFile.txt
A:\SWT [main ≡ +1 ~1 -1 ~]> git commit -m "Some modifications"
[main 05f6fec] Some modifications
 3 files changed, 2 insertions(+), 2 deletions(-)
 delete mode 100644 DeletedFile.txt
 create mode 100644 NewFile.txt
A:\SWT [main ↑1]> git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

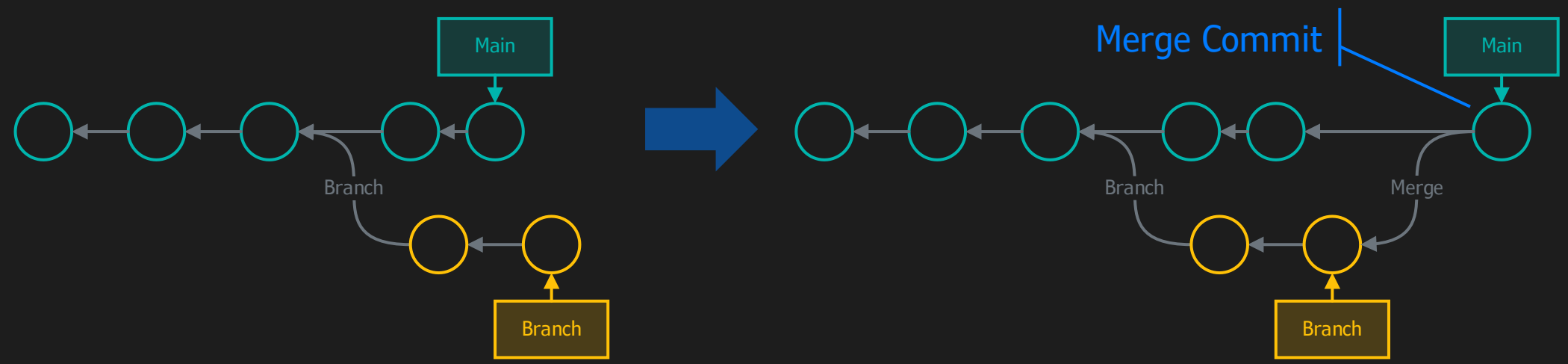
Befehle: Branching

- `git branch`
  - Listet alle lokalen Branches auf
- `git switch <branch>`
  - Wechselt in den angegebenen lokalen Branch
  - Mit Schalter `-c` wird ein neuer lokaler Branch erzeugt und in diesen gewechselt
- `git merge <branch>`
  - Führt alle Änderungen des angegebenen Branch einschließlich des gesamten Commit-Verlaufs seit der Trennung mit dem aktuellen Branch zusammen
  - »Fast Forward« bei linearem Verlauf: HEAD von aktuellem Branch wird auf HEAD von angegebenen Branch gesetzt
  - »Three-Way« bei divergierenden Verläufen: Zusammenführung in neuem »Merge Commit«
  - Schalter `--squash` reduziert alle Commits des angegebenen Branch zu einem einzigen Merge, der aber im Anschluss noch explizit committet werden muss
- `git diff <branch>`
  - Zeige alle Differenzen zwischen aktuellem und dem angegebenem Branch
- `git branch -d <branch>`
  - Lokalen Branch löschen

Merging



Fast Forward Merge

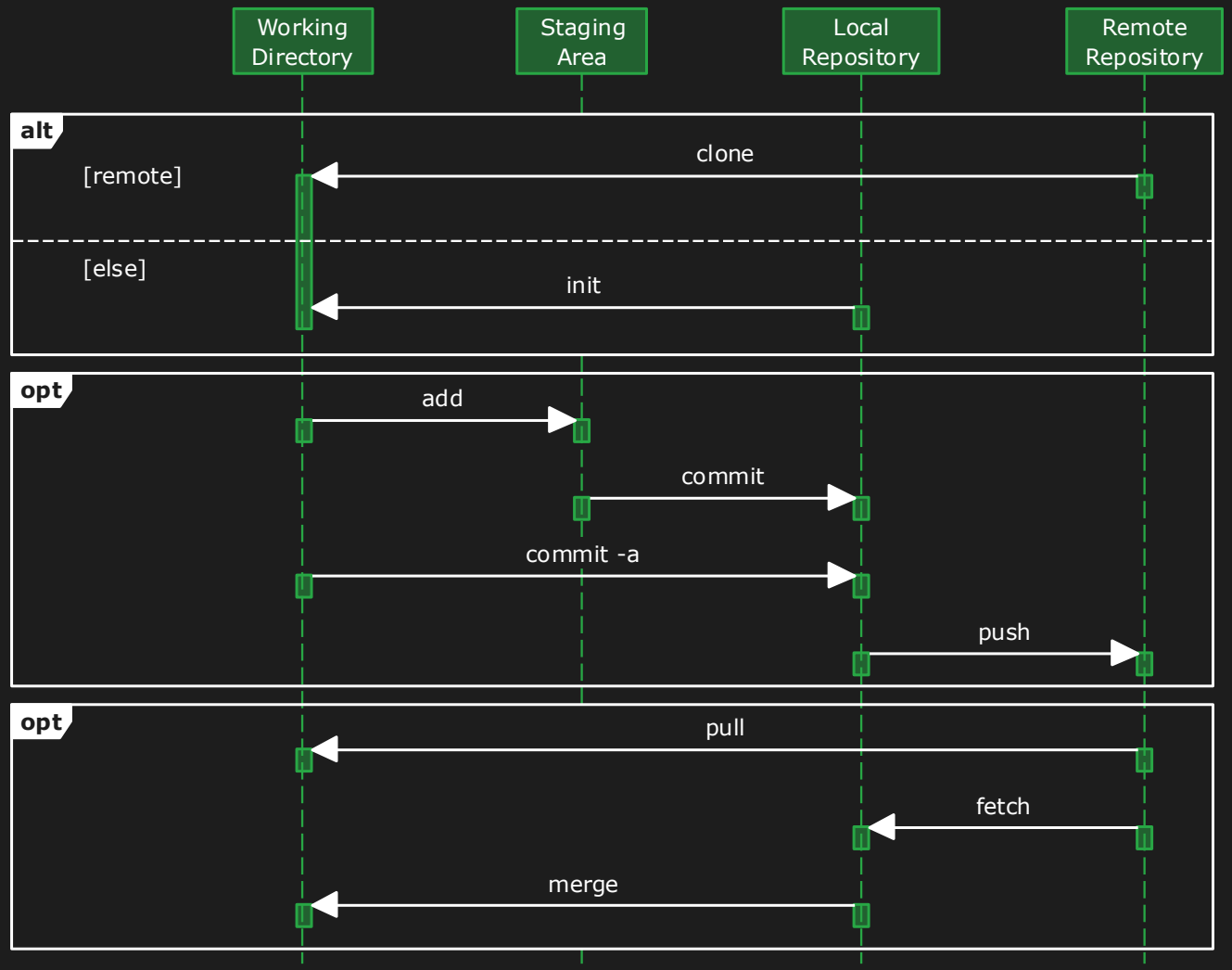


Three-Way Merge

Befehle: Remote Repository

- `git push <remote> <branch>`
  - Überträgt neue Commits des Branch (z.B. `main`) vom lokalen ins remote Repository (z.B. `origin`)
  - Nur möglich, wenn lokal auf letztem Stand, d.h. wenn remote ein »Fast Forward« möglich ist
  - Schalter `-u` setzt diesen als »Tracking Branch«
- `git fetch <remote> <branch>`
  - Lädt Änderungen des Branch aus dem remote ins lokale Repository
  - Das Arbeitsverzeichnis bleibt unverändert
- `git pull <remote> <branch>`
  - Kombiniertes Fetch & Merge
- Falls »Tracking Branch«, dann genügen `git push`, `git fetch` und `git pull`
- Erst Pull, dann Push wird auch als Synchronisation (»Sync«) bezeichnet

Übersicht: wichtige Repository-Befehle



Merge-Konflikte (1)

- Konflikt können beim Merge auftreten (auch während eines Pull)
- Git versucht, Konflikte möglichst selbst zu lösen
- Einige Konflikte, die Git nicht entscheiden kann, z.B.
  - Zwei Entwickelnde haben denselben Bereich einer Datei geändert
  - Entwickelnde Person hat Datei gelöscht, während andere entwickelnde Person diese ändert
- Diese Konflikte müssen manuell gelöst werden: dafür kann der gleiche Prozess wie bei allen Änderungen genutzt werden
- Merge und Konfliktbereinigung erfolgen immer in der lokalen Arbeitsumgebung
- Spezielle Merge Tools erleichtern die Arbeit
- Nach der vollständigen Konfliktbereinigung erfolgt das normale Commit & Push

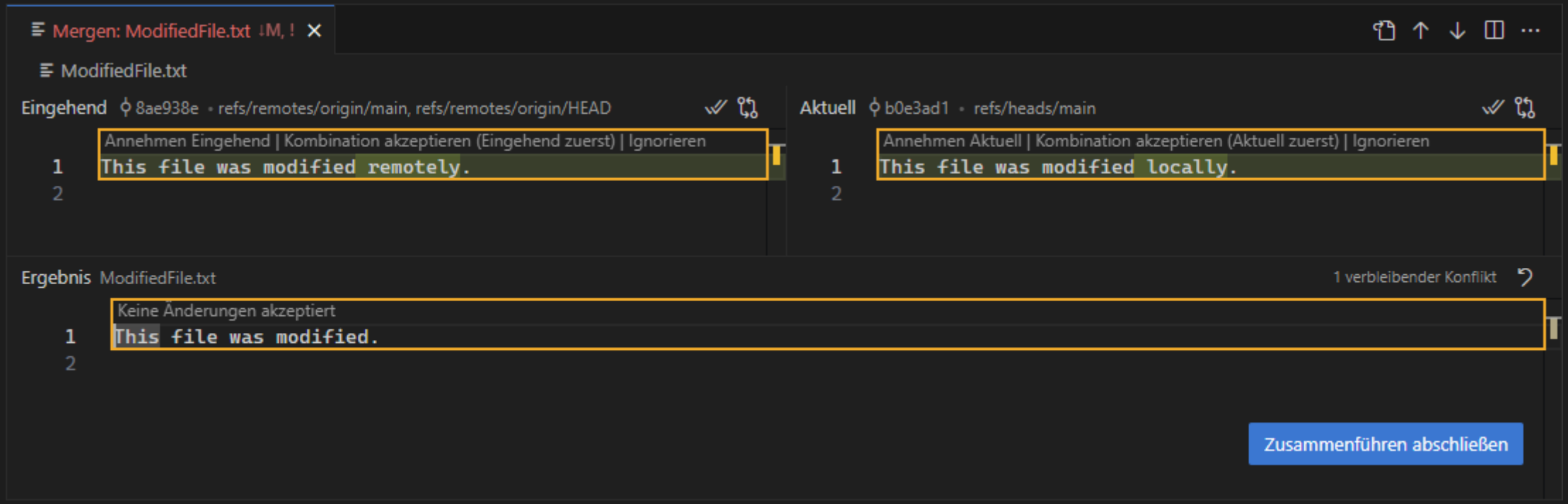
Merge-Konflikte (2)

```
A:\SWT [main ↑1]> git push
To https://git.fh-aachen.de/swt.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://git.fh-aachen.de/swt.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.

A:\SWT [main ↓1 ↑1]> git fetch

A:\SWT [main ↓1 ↑1]> git diff main origin/main
diff --git a/ModifiedFile.txt b/ModifiedFile.txt
index 3ab6499..f155e31 100644
--- a/ModifiedFile.txt
+++ b/ModifiedFile.txt
@@ -1 +1 @@
-This file was modified locally.
+This file was modified remotely.
```

Merge-Konflikte (3)

```
A:\SWT [main ↓1 ↑1]> git merge
Auto-merging ModifiedFile.txt
CONFLICT (content): Merge conflict in ModifiedFile.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Hinweis auf Merge-Konflikte



Konfliktbereinigung mit einem Merge Tool (hier Visual Studio Code)

Branching Strategy

- Strategie möglichst einfach halten, im Team kommunizieren & Einhaltung prüfen
- Strukturierung durch Workflows, z.B. Feature Branch Workflow
- Einheitliche Namenskonvention für Branches, z.B.
  - feature/feature-name
  - bugfix/description
  - users/username/description
- Remote Main: immer aktuell & hohe Qualität
  - Build sollten immer möglich sein, alle Tests sollten erfolgreich sein
  - Feature Branches: möglichst schnell Merge zurück in Main (Trunk-Based Development, TBD)
- Pull Requests bzw. Merge Requests vor Änderungen von remote Main
  - Code-Review-Prozess sollte definiert & eingehalten werden
  - Merge nur, wenn Review durchgeführt & Qualität bestätigt wurde
  - Studien zeigen: zwei Reviewer (neben Creator) sind optimal
  - Reviewer sollten regelmäßig aus dem ganzen Team wechseln

Feature Branch Workflow (lokal)

- Vorbereitung
  - `git switch main` (in lokalen Main wechseln)
  - `git pull` (aktuellen Stand von remote Main holen)
- In neuen lokalen Feature Branch wechseln
  - `git switch -c feature/new-feature`
- Arbeiten im lokalen Feature Branch: Add und Commit
- Regelmäßiges Synchronisieren des lokalen Feature Branch mit remote Main
  - `git pull origin main`
- Abschluss nach Fertigstellung des Feature
  - `git switch main` (in lokalen Main wechseln)
  - `git pull` (aktuellen Stand von remote Main holen)
  - `git merge feature/new-feature` (lokalen Feature Branch in lokalen Main mergen)
  - `git push` (lokalen Main nach Remote pushen)
  - `git branch -d feature/new-feature` (lokalen Feature Branch löschen)

Feature Branch Workflow (mit Pull Request)

- Nach Erstellung des Feature Branch diesen nach Remote pushen
  - `git push -u origin feature/new-feature`
- Zu geeigneten Zeitpunkten lokalen Fetaure Branch nach Remote pushen
  - `git push`
- Nach Fertigstellung des Feature
  - Pull Request in Remote erzeugen
  - Anpassungen aus Pull Request im lokalen Feature Branch einarbeiten und nach Remote pushen
- Abschluss nach vollständigem Pull Request
  - `git switch main` (in lokalen Main wechseln)
  - `git pull` (aktuellen Stand von remote Main holen)
  - `git pull origin feature/new-feature` (remote Feature Branch in lokalen Main mergen)
  - `git push` (lokalen Main nach Remote pushen)
  - `git branch -d feature/new-feature` (lokalen Feature Branch löschen)
  - `git push origin --delete feature/new-feature` (remote Feature Branch löschen)

Änderungen rückgängig machen

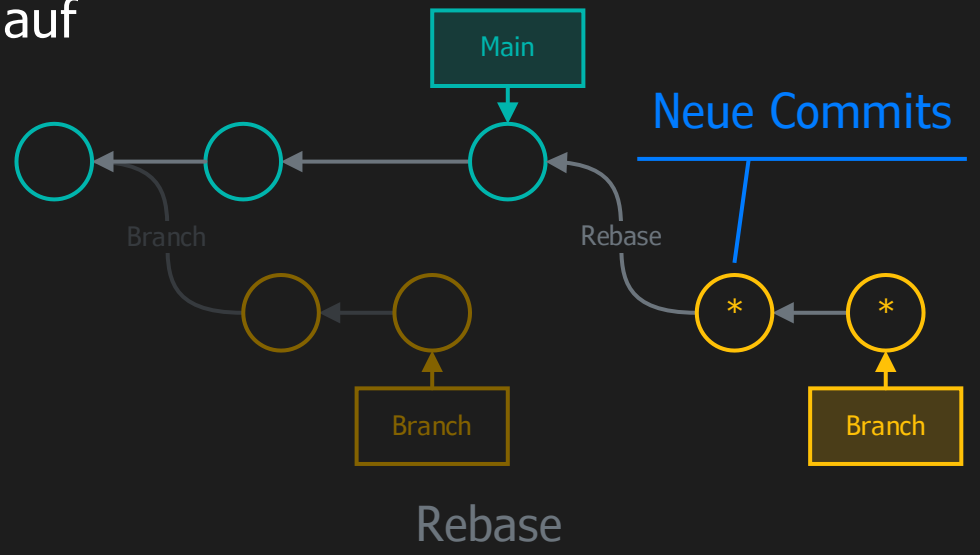
- `git restore <file>`
  - Datei im Arbeitsverzeichnis aus dem Index wiederherstellen
- `git reset <file>`
  - Datei im Index zurücksetzen; die Datei im Arbeitsverzeichnis wird nicht geändert
  - Ohne Pfadangabe werden alle Dateien im Index zurückgesetzt
- `git reset --hard`
  - Arbeitsverzeichnis & Index zurücksetzen
  - Achtung: alle Anpassungen im Arbeitsverzeichnis gehen verloren!
- `git clean`
  - Alle »Untracked«-Dateien (d.h. alle Dateien, die nicht unter Versionskontrolle von git stehen) werden aus Arbeitsverzeichnis gelöscht
  - Schalter `-n` für Was-wäre-wenn-Lauf (»Dry Run«), Schalter `-f` für erzwungene Durchführung
- `git revert <commit>`
  - Erzeugt einen neuen Commit, der alle Änderungen des angegebenen Commit zurücknimmt
  - Bevorzugt bei Remote Repository: dadurch bleibt der Commit-Verlauf konsistent

Befehle: Verlauf und Logs

- `git log`
  - Commit-Verlauf des aktuellen Branch
  - `-<limit>` (nur die letzten <limit> Commits)
  - `--oneline` (einzeilige Ausgabe)
  - `--graph` (mit Visualisierung des Branching)
  - `--stat` (nur statistische Ausgabe von Änderungen)
- `git log <file>`
  - Ausgabe nur der Commits, die die Datei geändert haben
- `git blame <file>`
  - Ausgabe, wer welche Änderung an der Datei vorgenommen hat

Weitere Konzepte

- Generierten Dateien sollten nicht unter Versionskontrolle gestellt werden, sondern immer nur lokal erzeugt werden (z.B. mit `.gitignore` ausschließen)
- Stash: Bereich, um Änderungen vorübergehend »zu parken«
- Rebase
  - Problem: Änderungen sollen gepusht werden, aber lokal nicht neuester Stand; eigentlich Merge & Commit nötig, aber dadurch neue, »unnötige« Commits
  - Durch Rebase werden Commits eines Branches an den letzten Stand des ursprünglichen Branches »angehängt«; ermöglicht das Beibehalten eines linearen Commit-Verlaufs
  - Achtung: der Commit-Verlauf wird dabei geändert! Nie auf öffentlichen Branches, weil »Breaking Changes«!
- Cherry-Picking
  - Integriert ausgewählte Commits aus mehreren Branches in den aktuellen Branch



Abschnitt W.2

Build Management mit Apache Maven

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Build Management

- Verschiedene Aufgaben eines Build Managements, z.B.
  - Konfigurieren des Systems
  - Verwaltung von Abhängigkeiten (Dependency Management)
  - Kompilieren & Linken des Quellcodes in der richtigen Reihenfolge
  - Zusammenstellung des Pakets (z.B. JAR) bzw. der Installationsroutine
  - Ausführung statischer Analysen
  - Ausführen von automatisierten Tests, insbesondere Komponententests
  - Ggf. Deployment in eine spezifische Umgebung
  - Erzeugen der technischen Dokumentation (z.B. Javadoc)
- Oft in integrierter Entwicklungsumgebung (IDE) enthalten
- Werkzeuge (Build Tools), z.B.
  - C/C++: Make
  - Java: Apache Ant, Apache Maven, Gradle
  - .NET: MSBuild

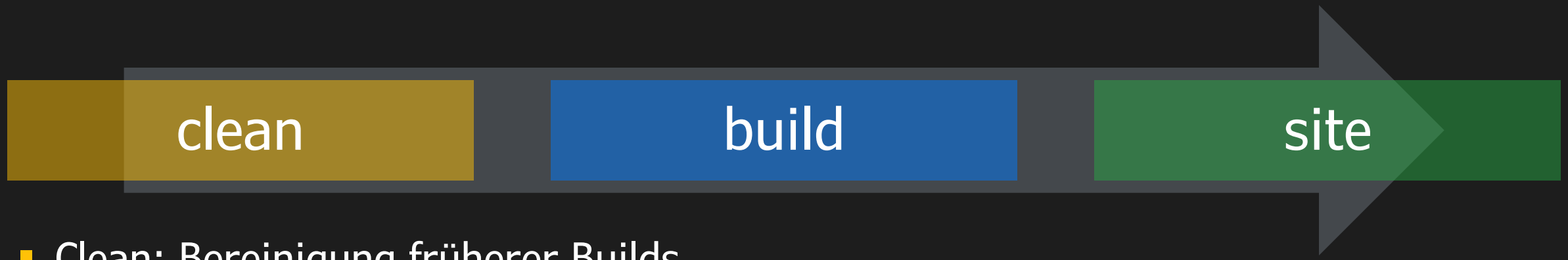
Apache Maven

- Seit 2002, Apache Software Foundation, in Java geschrieben
- Maven: jiddisch »Sammler des Wissens«
- Aktuelle Version 3.9.11 (Juli 2025)
- Tatsächlich mehr als nur Build Tool, z.B. auch Reports
- Orientiert sich an Convention over Configuration (CoC)
  - Konfigurationsdatei pom.xml (»Project Object Model«)
  - Durch festgelegte Konvention zur Namensgebung, Verzeichnisstruktur etc. müssen im Standard nur wenige Parameter konfiguriert werden
- Basisangaben: global eindeutige GAV-Parameter
  - groupId: Namespace des Anbieters, z.B. »de.phroh.swt«
  - artifactId: Name des Projekts, z.B. »demo«
  - version: Version des Build, z.B. »1.0«
  - Außerdem: Projekttyp, Entwickelnde, URL, Lizenzangaben etc.
- Konsolenanwendung, z.B. mvn compile
- Einbindung von externen Werkzeugen, z.B. SonarQube

Beispiel: pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>de.phroh.swt</groupId>
  <artifactId>demo</artifactId>
  <version>1.0</version>

  <name>Maven Demo</name>
  <url>https://www.phroh.de</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <java.version>21</java.version>
    <maven.compiler.source>${java.version}</maven.compiler.source>
    <maven.compiler.target>${java.version}</maven.compiler.target>
    <maven.compiler.release>${java.version}</maven.compiler.release>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <version>6.0.1</version>
      <scope>test</scope>
    </dependency>

    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-params</artifactId>
      <version>6.0.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.13.0</version>
      </plugin>
    </plugins>
  </build>

</project>
```

Apache Maven: Lebenszyklus



- Clean: Bereinigung früherer Builds
- Build: Erstellen des Projekts, Phasen über Plug-Ins
  - validate
  - compile
  - test
  - package
  - integration-test
  - verify
  - install (ins lokale Maven-Repository)
  - deploy (ins Remote-Maven-Repository)
- Site: Erstellen von Webseiten & Reports

Abschnitt W.3

Komponententests mit JUnit

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

JUnit

JUnit 6

- Test-Framework für Java
- Insbesondere Komponententests (Unit Tests)
- Anfänglich von Erich Gamma und Kent Beck entwickelt
- Aktuelle Version: 6.0.1 (Okt. 2025)
- Adaptionen für andere Programmiersprachen, z.B. NUnit (.NET), CppUnit (C++), QUnit (JavaScript)
- Auszeichnung des Test-Quellcode über Annotation @Test
  - Keine statischen oder privaten Methoden
  - Keine Parameter
  - Rückgabetyp void

Assertions

- Annahmen über erwartete Soll-Ergebnis (»expected«) in Bezug auf das tatsächliche Ist-Ergebnis (»actual«)
- Methoden in `org.junit.jupiter.api.Assertions.*`
- `assertTrue`, `assertFalse`: Test auf booleschen Wert
- `assertEquals`, `assertNotEquals`: Test auf Gleichheit
- `assertNull`, `assertNotNull`: Test auf Nullwert
- `assertSame`, `assertNotSame`: Test auf identische Klassen
- `assertArrayEquals`: Test, ob zwei Arrays die gleichen Elemente in der gleichen Reihenfolge besitzen
- `assertThrows`: Test auf das Werfen von Ausnahmen
  - Der Typ der Ausnahme kann spezifiziert werden, z.B. `NullPointerException()`

Empfehlungen

- Erstelle eine Testklasse für jede zu testende Klasse; Name der Testklasse = [Klassenname]Test
  - Beispiel: Klasse Math und MathTest
- Optimieren Test-Code auf Lesbarkeit, nicht auf Performanz
- Teste nur eine Annahme pro Test
- Nutze aussagekräftige Methodennamen, aus denen Test, Parameter & Annahme erkennbar sind
  - Schema: <methodUnderTest>_<scenario>_<expectedBehaviour>
  - »When I call method <methodUnderTest> with a <scenario> value, then it should do <expectedBehaviour>«

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Beispiel (1)

```
@Test
void add_with0As1stParam_shouldReturn2ndParam() {
    // Arrange
    final int m = new Random().nextInt();

    // Act
    final int actual = Math.add(0, m);

    // Assert
    assertEquals(m, actual);
}

@Test
void fibonacci_withNegativeNumberAsParam_shouldThrowNegativeArraySizeException() {
    // Act & assert
    assertThrows(NegativeArraySizeException.class, () -> Math.fibonacci(-1));
}
```

Parametrisierte Tests

- Seit JUnit 5 Jupiter: vereinfachte parametrisierte Tests
- Annotation der Testmethode: @ParameterizedTest
- Verschiedene vordefinierte Provider für Argumente, z.B.
  - @ValueSource
  - @CsvSource
  - @CsvFileSource
  - @EnumSource
  - @MethodSource
- Eigene ArgumentsProvider sind möglich

Beispiel (2)

```
@ParameterizedTest
@ValueSource(ints = { 0, 1, 2, 42 })
void fibonacci_withNonNegativeNumber_shouldReturnArrayOfThisLength(final int n) {
    // Act
    final int[] actual = Math.fibonacci(n);

    // Assert
    assertEquals(n, actual.length);
}

@ParameterizedTest
@CsvSource({ "1, 2, 3", "2, 1, 3", "42, 1, 43", "42, -84, -42" })
void add_withSamples_shouldReturnExpected(final int n, final int m, final int
expected) {
    // Act
    final int actual = Math.add(n, m);

    // Assert
    assertEquals(expected, actual);
}
```